

Syed mohsin shah

Cs221092 7B

Assignment : 1 P&DC

Q1: Alphabetical Sorting of a Huge String using OpenMP

PCAM Analysis

Partitioning: Split the string into equal parts.

Communication: Threads work independently, minimal communication.

Agglomeration: Each part is sorted locally.

Mapping: Each part is assigned to one thread (4 threads).

Algorithm

1. Read the input string.
2. Divide string into 4 equal parts.
3. Create 4 OpenMP sections.
4. Sort each part in parallel.
5. Merge sorted parts.
6. Display final sorted string.

Flowchart

Start → Input String → Divide into 4 Parts → Parallel Sort (4 Threads)
→ Merge Results → Display Sorted String → End

Program (C++ with OpenMP)

```
#include <bits/stdc++.h>
```

```
#include <omp.h>
```

```
using namespace std;
```

```
int main() {
```

```
    string s = "parallelcomputing";
```

```
    int n = s.size();
```

```
    int part = n / 4;
```

```
    string a[4];
```

```
    #pragma omp parallel sections
```

```
{
```

```
    #pragma omp section
```

```
    a[0] = s.substr(0, part);
```

```
    #pragma omp section
```

```

    a[1] = s.substr(part, part);

    #pragma omp section
    a[2] = s.substr(2*part, part);

    #pragma omp section
    a[3] = s.substr(3*part);
}

for(int i=0;i<4;i++)
    sort(a[i].begin(), a[i].end());

string result = a[0]+a[1]+a[2]+a[3];
sort(result.begin(), result.end());

cout << result;
}

```

Q2: Searching a 4-letter Word using OpenMP

PCAM Analysis

Partitioning: Divide string into 4 parts.

Communication: Shared flag to stop other threads.

Agglomeration: Each thread searches its chunk.

Mapping: Each chunk mapped to one thread.

Algorithm

1. Input string and word.
2. Divide string into 4 parts.
3. Start parallel sections.
4. Each thread searches its part.
5. If found, stop other threads.
6. Display result.

Flowchart

```

Start → Input Data → Divide String → Parallel Search
→ Word Found? → Yes → Stop Others → End
→ No → Continue → End

```

Program

```

#include <bits/stdc++.h>
#include <omp.h>
using namespace std;

int main() {

```

```

string s = "thisisaparallelstringsearch";
string word = "para";
bool found = false;

#pragma omp parallel sections shared(found)
{
    #pragma omp section
    if(!found && s.substr(0, s.size()/4).find(word) != string::npos) found = true;

    #pragma omp section
    if(!found && s.substr(s.size()/4, s.size()/4).find(word) != string::npos) found = true;

    #pragma omp section
    if(!found && s.substr(2*s.size()/4, s.size()/4).find(word) != string::npos) found = true;

    #pragma omp section
    if(!found && s.substr(3*s.size()/4).find(word) != string::npos) found = true;
}

if(found) cout << "Word Found";
else cout << "Word Not Found";
}

```

Q3: Multiplication of Three N×N Matrices

PCAM Analysis

Partitioning: Rows of matrices.

Communication: Shared matrices.

Agglomeration: Row-wise multiplication.

Mapping: Rows assigned to threads.

Algorithm

1. Input matrices A, B, C.
2. Multiply A and B in parallel.
3. Multiply result with C.
4. Display final matrix.

Flowchart

Start → Input Matrices → Parallel Multiplication
→ Final Matrix → End

Program

```

#include <omp.h>
#include <iostream>
using namespace std;

```

```

int main() {
    int N = 2;
    int A[2][2]={{1,2},{3,4}};
    int B[2][2]={{5,6},{7,8}};
    int C[2][2]={{1,1},{1,1}};
    int R[2][2]={0}, F[2][2]={0};

    #pragma omp parallel for
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            for(int k=0;k<N;k++)
                R[i][j]+=A[i][k]*B[k][j];

    #pragma omp parallel for
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            for(int k=0;k<N;k++)
                F[i][j]+=R[i][k]*C[k][j];

    cout << F[0][0];
}

```

Q4: Sum, Mean, and Largest Element using OpenMP

PCAM Analysis

Partitioning: Array split.

Communication: Shared variables.

Agglomeration: Reduction operations.

Mapping: Threads assigned tasks.

Algorithm

1. Input array.
2. Two threads calculate sum.
3. One thread calculates mean.
4. One thread finds largest value.
5. Display results.

Flowchart

Start → Input Array → Parallel Computation
 → Sum → Mean → Largest → End

Program

```

#include <omp.h>
#include <iostream>

```

```

using namespace std;

int main() {
    int arr[6]={2,4,6,8,10,12};
    int sum=0, max=arr[0];

    #pragma omp parallel for reduction(+:sum)
    for(int i=0;i<6;i++) sum+=arr[i];

    #pragma omp parallel for reduction(max:max)
    for(int i=0;i<6;i++) if(arr[i]>max) max=arr[i];

    float mean = sum/6.0;

    cout << sum << " " << mean << " " << max;
}

```

Q5: Static Scheduling

Explanation

Static scheduling divides loop iterations equally among threads before execution starts. Each thread gets fixed chunks of iterations.

Flowchart

Start → Set Threads → Static Distribution → Parallel Execution → End

Q6: Nested Parallelism

Explanation

Nested parallelism allows parallel regions inside other parallel regions. Outer threads create inner teams for further parallel execution.

Flowchart

Start → Outer Parallel Region → Inner Parallel Region → End